Isentropic

OpenJLS

JPEG-LS Lossless Encoder IP Core — Datasheet

1 Overview

OpenJLS is a JPEG-LS lossless encoder for FPGAs. It encodes single-component (grayscale) images at one pixel per clock with no external memory, streaming a standards-compliant JPEG-LS bitstream out over a standard ready/valid streaming handshake. Source code, license and the full verification suite are public at https://github.com/isentropic-fpga/OpenJLS. There's also a companion public repository with demo projects for OpenJLS at https://github.com/isentropic-fpga/OpenJLS-Demos.

Table 1: Key specifications.

Standard	JPEG-LS (ISO/IEC 14495-1, ITU-T T.87)
Compression	Lossless only
Components	Single (grayscale)
Pixel depth	8–16 bits (`BITNESS`)
Image size	Up to 65535 × 65535 px
Throughput	1 pixel / clock
Latency	14 clock cycles
f_{max}	∼280 MHz at 12 bit (Xilinx `xczu7eg-1`)
Interface	Ready/valid streaming (AXI4-Stream / Avalon-ST compatible)
Memory	On-chip line buffer, one image row; no external RAM
Resources	∼6.4k LUT, ∼2.0k FF + BRAM scaling with image width
Portability	Vendor-agnostic VHDL

2 Revision and contact

Table 2: Revision history.

Version	Date	Notes
1.0	2026-06	Initial release.
1.1	2026-07	Packaged IPs for Vivado.
1.2	2026-07	Isentropic branding; cores repackaged under the `isentropic` vendor.
1.3	2026-09	Frequency increased ∼15%, LUT reduction ∼16%, BRAM up 1 tile.

License. OpenJLS is dual-licensed: free under GPL v3 for any GPL-compliant use, or under a commercial license for proprietary products without GPL obligations. Evaluation — clone, simulate, synthesize, test — is unrestricted; a commercial license is required only to ship a product.
OpenJLS is developed and maintained by Isentropic. For commercial licensing, technical questions, or collaboration inquiries, contact **contact@isentropic.com.br**.

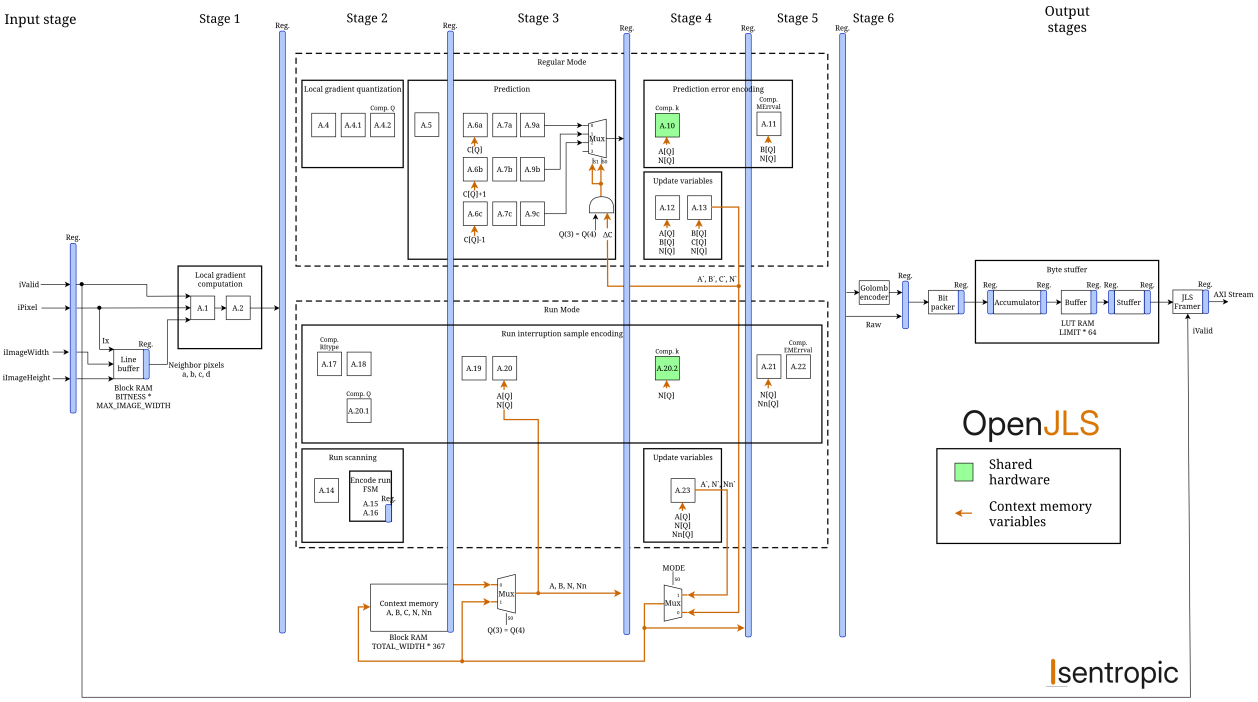
3 Functional description

The core is a fully pipelined encoder with a latency of 14 clock cycles: an input register, six processing stages (gradient computation, context modeling, MED prediction with bias correction, Golomb–Rice / run-length coding) and three internally registered output stages (bit packer, byte stuffer, framer).

End-of-image is derived internally from the dimensions provided at the ports `iImageWidth` and `iImageHeight`. The core never stalls — neither at EOI (end of image) nor at EOL (end of line) — which allows processing multiple images back-to-back. The only exception is when the downstream stalls long enough for the byte-stuffer buffer to fill, which in turn stalls the upstream.

While the pipeline latency is 14 cycles, the core does not emit an output beat every cycle: the output stages accumulate compressed bits and assert `oValid` only once a full `OUT_WIDTH` word is ready. As the bitstream is compressed, these output beats are infrequent relative to the one-pixel-per-clock input.

Figure 1: Encoder pipeline architecture.



A full-resolution version of this diagram is available in the repository.

4 Generics

The core is configured by the four generics in Table 3.

Table 3: Configuration generics.

Generic	Range	Description
`BITNESS`	8–16	Pixel bit depth.
`MAX_IMAGE_WIDTH`	4–65535	Largest width supported; sets line-buffer depth.
`MAX_IMAGE_HEIGHT`	1–65535	Largest height supported.
`OUT_WIDTH`	48–1024	Output bus width in bits; multiple of 8.

`MAX_IMAGE_WIDTH` and `MAX_IMAGE_HEIGHT` fix the *compile-time* maximum image size: they size the on-chip line buffer and the dimension counters, so a larger maximum costs more BRAM. They do not select the size of a given image — the dimensions of each encoded image are chosen at *run time* through the `iImageWidth` and `iImageHeight` ports (Table 4), which accept any value from the minimum up to the configured maximum.

5 Ports

All ports are synchronous to the rising edge of `iClk` — the core is a single clock domain. The streaming ports use a plain ready/valid handshake; in Table 4 the **AXIS** column gives the one-to-one AXI4-Stream signal mapping for that ecosystem (Avalon-ST maps the same way at `readyLatency` = 0).

Table 4: Port list and AXI4-Stream mapping.

Port	Dir	Width	AXIS	Role
`iClk`	in	1	—	Clock.
`iRst`	in	1	—	Synchronous reset, active high; latches dimensions.
`iImageWidth`	in	16	—	Image width, pixels (configuration).
`iImageHeight`	in	16	—	Image height, pixels (configuration).
`iValid`	in	1	`TVALID`	Pixel valid.
`iPixel`	in	`BITNESS`	`TDATA`	Pixel data.
`oReady`	out	1	`TREADY`	Input ready.
`oData`	out	`OUT_WIDTH`	`TDATA`	Bitstream beat.
`oValid`	out	1	`TVALID`	Output valid.
`oKeep`	out	`OUT_WIDTH/8`	`TKEEP`	Valid-byte mask on final beat.
`oLast`	out	1	`TLAST`	End of image.
`iReady`	in	1	`TREADY`	Downstream ready.

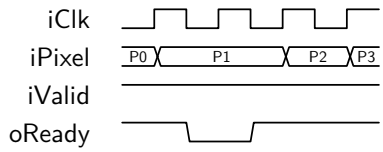
IMPORTANT:

- `iImageWidth` and `iImageHeight` are only sampled while `iRst` is asserted (`'1'`).
- If an invalid value is provided to `iImageWidth`, that is `iImageWidth > MAX_IMAGE_WIDTH` or `iImageWidth < 4`, the encoder uses `MAX_IMAGE_WIDTH` as the image width.
- If an invalid value is provided to `iImageHeight`, that is `iImageHeight > MAX_IMAGE_HEIGHT` or `iImageHeight < 1`, the encoder uses `MAX_IMAGE_HEIGHT` as the image height.
- If you don't intend to change the image dimensions at run-time, wire both inputs to `0` so that the maximum values are used.
- Both ports are a fixed 16 bits regardless of the `MAX_IMAGE_*` generics — every representable value is checked, so any out-of-range value is caught and replaced by the maximum.

6 Timing and protocol

Pixel input. Pixels are streamed in scan order — the first row left to right, then the second row, and so on — each an unsigned value on `iPixel`. A pixel transfers on every cycle where `iValid` and `oReady` are both high. The core asserts `oReady` continuously and deasserts it only when the FIFO in byte stuffer fills under downstream backpressure. Figure 2 shows a transfer with a brief downstream stall.

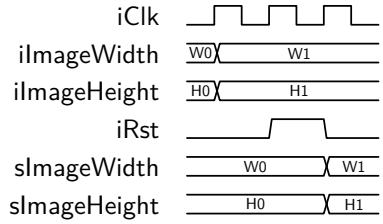
Figure 2: Pixel-input handshake.



In Figure 2, `P2` is held flat across the stall and accepted when `oReady` returns high; no pixel is dropped while `oReady` is low.

Reset and configuration. `iRst` is level-sensitive (hold it high for at least one clock) and clears the pipeline. The image dimensions are captured only while `iRst` is high: present `iImageWidth` and `iImageHeight`, hold them stable across the reset pulse, and they latch on the rising edge (Figure 3). A reset is therefore needed only to capture the dimensions — before the first image and whenever the image size changes; with the size unchanged, images encode back-to-back.

Figure 3: Reset and dimension capture.



In Figure 3, the new size (`W1`/`H1`) is driven for several cycles, then `iRst` pulses high; the dimensions register on the rising edge while `iRst` is high.

Output. Beats appear on `oData`/`oValid` subject to `iReady`; `oLast` marks the final beat (EOI) and `oKeep` flags its valid bytes. The bitstream is byte-serial and MSB-first: the earliest byte occupies the most-significant byte of `oData`. Pipeline latency is 14 clock cycles.

7 Resources and performance

Characterized on Xilinx Zynq UltraScale+ `xczu7eg-fbvb900-1-e`, Vivado 2025.2, using the generics `BITNESS = 12` (unless stated otherwise) and `OUT_WIDTH = 64`, RTL-only with no floorplanning. f_{max} is the true maximum, found by over-constraining the clock until timing fails. At one pixel per clock, ∼280 MHz is ∼280 Mpixel/s.
Figure 4 and Table 5 report f_{max} across `MAX_IMAGE_WIDTH`; Figure 5 and Table 6 report the resource usage; Figure 6 and Table 7 report both across `BITNESS`.

Figure 4: Maximum frequency vs maximum image width.

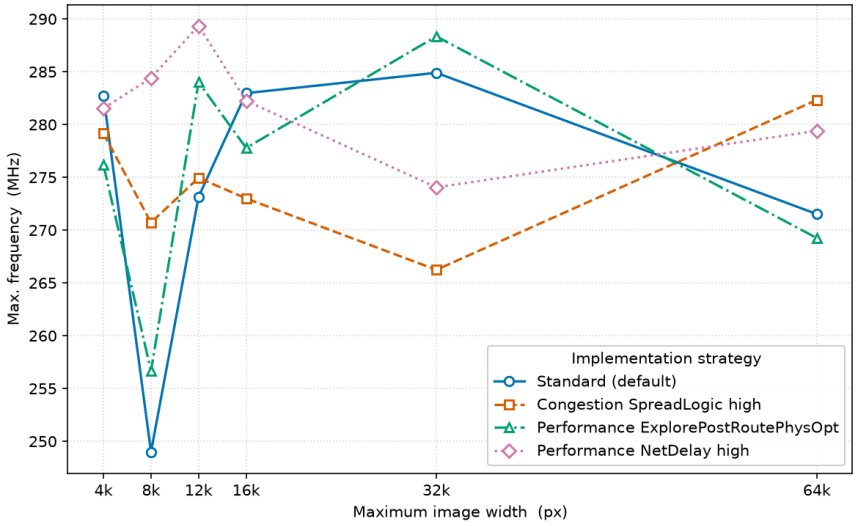


Table 5: Maximum frequency by maximum image width and strategy.

`MAX_IMAGE_WIDTH`	Default	EPRPO	NetDelay	Congestion
4096	**282.7**	276.2	281.5	279.2
8192	249.0	256.7	**284.4**	270.7
12288	273.1	284.0	**289.4**	274.9
16384	**283.0**	277.8	282.2	273.0
32768	284.9	**288.4**	274.1	266.2
65535	271.5	269.2	279.4	**282.3**

Figure 5: Resource usage vs maximum image width.

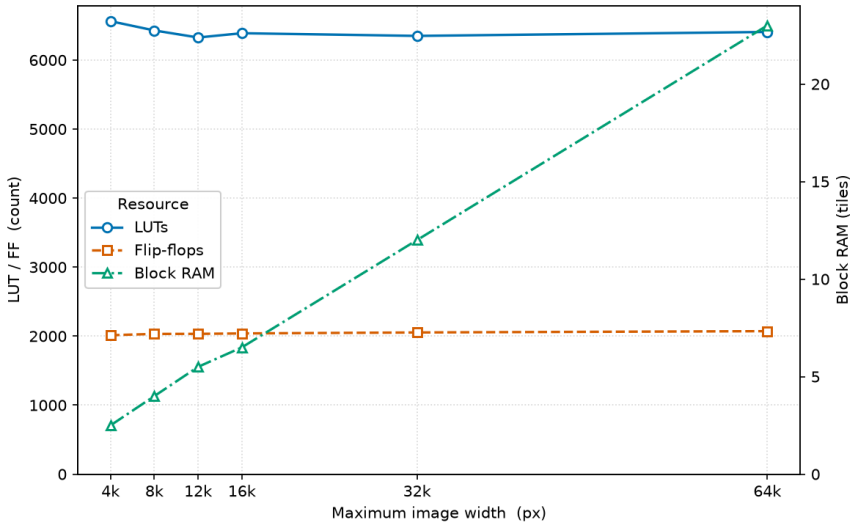


Table 6: Resource usage by maximum image width.

MAX_IMAGE_WIDTH	LUT	FF	BRAM
4096	6563	2013	2.5
8192	6429	2032	4.0
12288	6327	2033	5.5
16384	6389	2039	6.5
32768	6350	2053	12.0
65535	6407	2072	23.0

In Table 5 frequencies are in MHz and the best strategy per row is in bold; the strategies are `Performance_ExplorePostRoutePhysOpt` (EPRPO), `Performance_NetDelay_high` (NetDelay) and `Congestion_SpreadLogic_high` (Congestion). The design is congestion-bound, so the winning strategy shifts with image size; a given netlist is deterministic, but the per-size winner is placement-sensitive and can shift when the netlist changes, so the best-of band (∼282–289 MHz) is the headline figure; the per-size spread across strategies is 7–22 MHz, so a default run lands close to it at most sizes. Table 6 is for the default strategy; utilization is near-identical across strategies. Logic is essentially constant with image size; only the line-buffer BRAM scales with width and pixel depth.

Figure 6: Maximum frequency vs pixel bit depth (`MAX_IMAGE_WIDTH = 12288`).

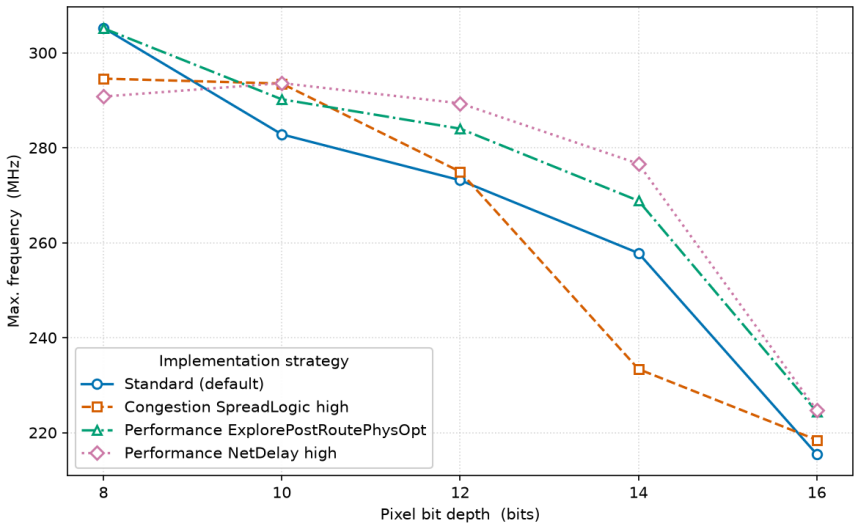


Table 7: Maximum frequency and resource usage by pixel bit depth (`MAX_IMAGE_WIDTH = 12288`).

BITNESS	Default	EPRPO	NetDelay	Congestion	LUT	FF	BRAM
8	**305.3**	305.2	290.8	294.6	5319	1772	3.5
10	282.8	290.2	**293.6**	293.5	5908	1902	4.0
12	273.1	284.0	**289.4**	274.9	6327	2033	5.5
14	257.8	268.8	**276.6**	233.4	6845	2153	6.5
16	215.4	224.4	**224.7**	218.4	7601	2287	7.5

Best-of f_{max} falls from ∼305 MHz at 8 bits to ∼277 MHz at 14 bits and ∼225 MHz at 16 bits. The byte stuffer is off the critical path at every depth; the limit is the context RAM read into the prediction-error register, whose carry chains widen with pixel depth. Resources are for the default strategy.

8 Integration example

```
u_enc : entity work.openjls_top
  generic map (
    BITNESS => 8,
    MAX_IMAGE_WIDTH => 4096,
    MAX_IMAGE_HEIGHT => 4096,
    OUT_WIDTH => 64 )
  port map (
    iClk => clk,
    iRst => rst,
    iImageWidth => img_w, -- latched on reset
    iImageHeight => img_h, -- latched on reset
    iValid => s_valid, -- pixel stream (AXIS slave)
    iPixel => s_data,
    oReady => s_ready,
    oData => m_tdata, -- bitstream (AXIS master)
    oValid => m_tvalid,
    oKeep => m_tkeep,
    oLast => m_tlast,
    iReady => m_tready );
```

Being a single entity with standard ready/valid ports, `openjls_top` can equally be dropped onto a block diagram and wired there, instead of instantiating it in HDL. The dimension ports are fixed at 16 bits (rather than sized from the generics) because Vivado's block-design port-width evaluator only handles literal arithmetic in port expressions.

Xilinx IP cores

For the Vivado flow the core ships the following pre-packaged IPs

IP Catalog name	**Interfaces**
OpenJLS Encoder (Native)	Plain Ready/Valid
OpenJLS Encoder (AXI4-Stream)	AXI-Stream for the data, native pins for control
OpenJLS Encoder (AXI4-Stream + AXI4-Lite)	AXI-Stream for the data, AXI-Lite for control

To use simply add `Sources/Xilinx/ip_repo/` to the project's IP repositories and the three cores appear in the IP Catalog, ready to drop onto a block design.

AXI4-Lite register map

The AXI4-Lite variant replaces the native control pins with a register bank (32-bit registers, word-aligned offsets):

Offset	Name	Access	Contents
0x00	ID	RO	ASCII "OJLS" (0x4F4A4C53)
0x04	VERSION	RO	0x00MMmmpp (major/minor/patch)
0x08	CAPS	RO	[7:0] BITNESS, [15:8] output bytes per beat (OUT_WIDTH/8)
0x0C	MAXDIM	RO	[15:0] MAX_IMAGE_WIDTH, [31:16] MAX_IMAGE_HEIGHT
0x10	WIDTH	RW	[15:0] image width (clamped on write)
0x14	HEIGHT	RW	[15:0] image height (clamped on write)
0x18	CTRL	WO	[0] APPLY — self-clearing, pulses the core reset
0x1C	STATUS	RO	[0] BUSY (reset pulse active), [1] pixel-stream TREADY mirror

The core samples the dimensions only while its reset is high, so reconfiguration is: write WIDTH/HEIGHT, set CTRL.APPLY. APPLY pulses the core reset for one clock while the registers are held stable; the AXI-Lite endpoint itself stays on `aresetn` only, so the bus never disappears mid-reconfigure. While the pulse is active STATUS.BUSY reads 1 and the pixel stream's TREADY is held low, so a stream started too early stalls instead of losing pixels — WIDTH/HEIGHT/CTRL writes while BUSY are dropped. **APPLY aborts any image in flight — only apply between images.** Back-to-back images of unchanged dimensions need no APPLY.

WIDTH/HEIGHT writes are merged per WSTRB and clamped to the core's rule (out-of-range values become the MAX generic), so a readback always returns the value the core will actually use. Writes to RO or unmapped offsets are acknowledged (OKAY) and dropped; unmapped reads return zero.

For driver code there is a C header in the repository, ojls_regs.h.

9 Conformance and verification

Verified by simulation with NVC using a layered, self-checking suite run at both RTL and post-synthesis (gate-level) stages:

- **OSVVM** — Per-module tests against independent T.87-derived reference models, top-level control-plane stress (reset injection, backpressure, back-to-back images, dimension fallback), and AXI wrapper tests (byte-exact AXI4-Stream pass-through at 8- and 12-bit, AXI4-Lite register map, live reconfiguration, mid-image abort), all with requirements tracking.
- **Coverage** — OSVVM functional coverage with NVC structural code coverage (99%+ statements).
- **Golden model** — Byte-exact vs the independent CharLS C++ encoder over 287 real and synthetic images, plus ISO/IEC 14495-1 vectors.
- **Design contracts** — Embedded PSL assertions (ready/valid handshake protocol, internal handshakes) on every run.
- **Post-synthesis** — The stress test and a golden-model subset re-run on the synthesized netlist.
- **Hardware-in-the-loop** — The full 287-image corpus encoded end-to-end on a PYNQ-Z2 FPGA and byte-compared against CharLS on real silicon. Produced in the demos repository OpenJLS-Demos by the EncodeOverEthernet project.

Table 8 is the latest regression snapshot.

Table 8: Latest regression snapshot.

Suite	Status	Test/Cov	Summary
NVC code coverage	info	99.8%	Per-DUT-file statement breakdown
OSVVM suite	PASS	100%	106 tests, 7 303 829 affirmations (module + top + AXI wrappers)
Golden model	PASS	100%	287/287 images byte-exact vs CharLS
Post-synth OSVVM	PASS	100%	Control-plane stress on the gate-level netlist
Post-synth golden model	PASS	100%	156/156 images byte-exact vs CharLS
Hardware-in-the-loop	PASS	100%	287/287 images byte-exact vs CharLS on PYNQ-Z2 silicon

References. ITU-T T.87 / ISO/IEC 14495-1; Y. M. Mert, *Key Architectural Optimizations for Hardware Efficient JPEG-LS Encoder*, IEEE 2018; CharLS; open-logic.